

Міністерство освіти і науки України
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ім. Богдана Хмельницького

Факультет Обчислювальної техніки, інтелектуальних та
управляючих систем

Лабораторна робота №1
по дисципліні «Системи штучного інтелекту»
Тема: «Умовні, циклічні конструкції, функції та рекурсія у мові Python»

Виконала:
Студентка 3-го курсу
групи ЗКН-23
Костенко Віталіна Олександрівна

Зміст

1. Мета роботи
2. Завдання
3. Завдання 1: Обчислення складної функції
 - 3.1 Постановка задачі
 - 3.2 Код програми
 - 3.3 Результати тестування
4. Завдання 2: Циклічне обчислення з ключовим значенням
 - 4.1 Постановка задачі
 - 4.2 Код програми
 - 4.3 Результати тестування
5. Завдання 3: Обчислення суми ряду
 - 5.1 Постановка задачі
 - 5.2 Код програми
 - 5.3 Результати тестування
6. Висновки

1. Мета роботи

Набути практичних навичок програмування на мові Python: робота з математичними функціями, умовними операторами, циклами, рекурсією, обробкою помилок та графічним інтерфейсом tkinter.

2. Завдання

1. Скласти програму для знаходження значень складної функції, враховуючи її область визначення. Програма повинна виконуватись в окремому вікні, запитувати аргумент, показувати підфункцію, за якою обчислюється значення складної функції. Якщо змінна входить до складу аргументу тригонометричної функції, то (якщо можливо) її значення повинно вводиться у градусах. Якщо значення функції знайти неможливо, програма повинна повідомити про причину цього.
2. Модифікувати програму завдання №1 так, щоб програма працювала, поки користувач не введе деякого "ключового" значення для змінної x . "Ключове" значення повинно задаватися користувачем під час роботи програми.
3. Скласти програму знаходження значення суми n елементів виразу. Програма повинна містити 3 функції: перша — за допомогою циклу `for`, друга — `while`, третя — рекурсії. Описані функції повинні повертати логічне значення, яке повідомляє про успішність обчислення, і результат.

3. Завдання 1: Обчислення складної функції

3.1 Постановка задачі

Обчислити значення складної функції (Варіант 2):

$$y = \begin{cases} 1 / (\operatorname{ctg}(3 - x) - \arcsin(x)), & \text{при } x < 2 \\ \cos(1/(3 - x)) / \ln(2x - 5), & \text{при } x > 2 \end{cases}$$

Область визначення:

- Гілка $x < 2$: $|x| \leq 1$ (для $\arcsin$), $\sin(3-x) \neq 0$, $\operatorname{ctg}(3-x) - \arcsin(x) \neq 0$
- Гілка $x > 2$: $x \neq 3$ (ділення на $3-x$), $2x-5 > 0$ ($x > 2.5$), $\ln(2x-5) \neq 0$ ($x \neq 3$)
- Функція не визначена при $x = 2$

3.2 Код програми (task1.py)

```
"""
```

Лабораторна робота №1. Завдання 1. Варіант 2.

Обчислення значень складної функції з урахуванням області визначення.

$$y = \begin{cases} 1 / (\operatorname{ctg}(3-x) - \arcsin(x)), & \text{при } x < 2 \\ \cos(1/(3-x)) / \ln(2x-5), & \text{при } x > 2 \end{cases}$$

Тригонометричні аргументи вводяться в градусах.

"""

```
import math
import tkinter as tk
from tkinter import messagebox
```

```
def deg_to_rad(degrees):
    return math.radians(degrees)
```

```
def ctg(x):
    """Котангенс: cos(x)/sin(x)"""
    sin_val = math.sin(x)
    if sin_val == 0:
        raise ValueError("Знаменник дорівнює нулю: sin(x) = 0 у котангенсі")
    return math.cos(x) / sin_val
```

```
def compute_y(x):
    """
    Обчислює значення складної функції для варіанту 2.
    Повертає (значення, опис_підфункції) або кидає виняток з причиною.
    """
    if x == 2:
        raise ValueError("Функція не визначена при x = 2 (x не належить жодній гілці)")

    if x < 2:
        subfunc = "y = 1 / (ctg(3 - x) - arcsin(x))"

        if abs(x) > 1:
            raise ValueError(f"arcsin(x) не визначений: |x| = {abs(x)} > 1")

        arg_ctg = 3 - x
        sin_arg = math.sin(arg_ctg)
        if abs(sin_arg) < 1e-15:
            raise ValueError(f"ctg(3 - x) не визначений: sin(3 - x) = 0 при x = {x}")

        ctg_val = ctg(arg_ctg)
        arcsin_val = math.asin(x)

        denominator = ctg_val - arcsin_val
        if abs(denominator) < 1e-15:
            raise ValueError(
                f"Знаменник дорівнює нулю: ctg(3 - x) - arcsin(x) = 0 при x = {x}"
            )

        y = 1.0 / denominator
        return y, subfunc
```

```

else: # x > 2
    subfunc = "y = cos(1/(3 - x)) / ln(2x - 5)"

    if x == 3:
        raise ValueError("Ділення на нуль: 3 - x = 0 при x = 3")

    ln_arg = 2 * x - 5
    if ln_arg <= 0:
        raise ValueError(
            f"ln(2x - 5) не визначений: 2x - 5 = {ln_arg} <= 0 при x = {x}"
        )
    if abs(ln_arg - 1) < 1e-15:
        raise ValueError(
            f"Знаменник дорівнює нулю: ln(2x - 5) = ln(1) = 0 при x = {x}"
        )

    cos_arg = 1.0 / (3 - x)
    cos_val = math.cos(cos_arg)
    ln_val = math.log(ln_arg)

    y = cos_val / ln_val
    return y, subfunc

def run_gui():
    """Запуск програми в окремому вікні (tkinter)."""

    def calculate():
        try:
            x_str = entry_x.get().strip()
            if not x_str:
                messagebox.showwarning("Увага", "Введіть значення x!")
                return

            x = float(x_str)

            is_degrees = var_degrees.get()
            if is_degrees:
                x_calc = deg_to_rad(x)
                input_info = f"x = {x}° ({x_calc:.6f} рад)"
            else:
                x_calc = x
                input_info = f"x = {x}"

            y, subfunc = compute_y(x_calc)
            result_text.set(
                f"Вхідні дані: {input_info}\n"
                f"Підфункція: {subfunc}\n"
                f"Результат: y = {y:.10f}"
            )

        except ValueError as e:
            result_text.set(f"Помилка: {e}")
        except Exception as e:
            result_text.set(f"Непередбачена помилка: {e}")

```

```

root = tk.Tk()
root.title("Завдання 1 – Варіант 2")
root.geometry("550x300")
root.resizable(False, False)

frame = tk.Frame(root, padx=20, pady=20)
frame.pack(fill=tk.BOTH, expand=True)

tk.Label(frame, text="Введіть значення x:").grid(row=0, column=0, sticky="w")
entry_x = tk.Entry(frame, width=20)
entry_x.grid(row=0, column=1, padx=5)

var_degrees = tk.BooleanVar(value=False)
tk.Checkbutton(
    frame,
    text="Аргумент у градусах (для триг. функцій)",
    variable=var_degrees,
).grid(row=1, column=0, columnspan=2, sticky="w", pady=5)

tk.Button(frame, text="Обчислити", command=calculate, width=15).grid(
    row=2, column=0, columnspan=2, pady=10
)

result_text = tk.StringVar(value="")
tk.Label(
    frame,
    textvariable=result_text,
    justify=tk.LEFT,
    wraplength=500,
    font=("Consolas", 10),
).grid(row=3, column=0, columnspan=2, sticky="w")

root.mainloop()

if __name__ == "__main__":
    run_gui()

```

3.3 Результати тестування

х	Гілка	Результат
0.50	$x < 2$	$y = -0.5369857229$
-0.50	$x < 2$	$y = 0.3131639800$
1.00	$x < 2$	$y = -0.4929863130$
0.00	$x < 2$	$y = -0.1425465431$
2.50	$x > 2$	Помилка: $\ln(2x-5) = \ln(0)$ ≤ 0
4.00	$x > 2$	$y = 0.4918043530$
5.00	$x > 2$	$y = 0.5452727037$
10.00	$x > 2$	$y = 0.3655077221$
2.00	—	Помилка: $x = 2$, не

		визначена
3.00	$x > 2$	Помилка: $3-x = 0$, ділення на нуль
1.50	$x < 2$	Помилка: $ x = 1.5 > 1$, $\arcsin$ не визначений

Програма коректно обробляє всі граничні випадки: ділення на нуль, вихід за область визначення $\arcsin$, невизначений логарифм.

4. Завдання 2: Циклічне обчислення з ключовим значенням

4.1 Постановка задачі

Модифікувати програму завдання 1 так, щоб вона працювала в циклі, поки користувач не введе деякого "ключового" значення для x . "Ключове" значення задається користувачем на початку роботи програми.

4.2 Код програми (task2.py)

```
"""
Лабораторна робота №1. Завдання 2. Варіант 2.
Модифікація програми завдання 1: цикл до введення "ключового" значення  $x$ .

Програма працює в циклі, обчислюючи значення складної функції,
поки користувач не введе задане "ключове" значення  $x$ .
"Ключове" значення задається користувачем на початку роботи програми.
"""

import math
import tkinter as tk
from tkinter import messagebox

def deg_to_rad(degrees):
    return math.radians(degrees)

def ctg(x):
    sin_val = math.sin(x)
    if sin_val == 0:
        raise ValueError("Знаменник дорівнює нулю:  $\sin(x) = 0$  у котангенсі")
    return math.cos(x) / sin_val

def compute_y(x):
    if x == 2:
        raise ValueError("Функція не визначена при  $x = 2$  ( $x$  не належить жодній гілці)")

    if x < 2:
        subfunc = "y = 1 / (ctg(3 - x) - arcsin(x))"

        if abs(x) > 1:
            raise ValueError(f"arcsin(x) не визначений:  $|x| = \{abs(x)\} > 1$ ")

        arg_ctg = 3 - x
        sin_arg = math.sin(arg_ctg)
        if abs(sin_arg) < 1e-15:
            raise ValueError(f"ctg(3 - x) не визначений:  $\sin(3 - x) = 0$  при  $x = \{x\}$ ")

        ctg_val = ctg(arg_ctg)
        arcsin_val = math.asin(x)
        denominator = ctg_val - arcsin_val
        if abs(denominator) < 1e-15:
            raise ValueError(
```



```

        f"Знаменник дорівнює нулю:  $\text{ctg}(3 - x) - \arcsin(x) = 0$  при  $x = \{x\}$ "
    )

    y = 1.0 / denominator
    return y, subfunc
else:
    subfunc = "y =  $\cos(1/(3 - x)) / \ln(2x - 5)$ "

    if x == 3:
        raise ValueError("Ділення на нуль:  $3 - x = 0$  при  $x = 3$ ")

    ln_arg = 2 * x - 5
    if ln_arg <= 0:
        raise ValueError(
            f" $\ln(2x - 5)$  не визначений:  $2x - 5 = \{ln\_arg\} \leq 0$  при  $x = \{x\}$ "
        )
    if abs(ln_arg - 1) < 1e-15:
        raise ValueError(
            f"Знаменник дорівнює нулю:  $\ln(2x - 5) = \ln(1) = 0$  при  $x = \{x\}$ "
        )

    cos_arg = 1.0 / (3 - x)
    cos_val = math.cos(cos_arg)
    ln_val = math.log(ln_arg)

    y = cos_val / ln_val
    return y, subfunc


def run_gui():
    key_value = [None]
    history = []

    def set_key():
        try:
            val = float(entry_key.get().strip())
            key_value[0] = val
            lbl_key_status.config(text=f"Ключове значення встановлено: {val}")
            entry_key.config(state="disabled")
            btn_set_key.config(state="disabled")
            entry_x.config(state="normal")
            btn_calc.config(state="normal")
        except ValueError:
            messagebox.showwarning("Увага", "Введіть коректне числове значення!")

    def calculate():
        if key_value[0] is None:
            messagebox.showwarning("Увага", "Спочатку задайте ключове значення!")
            return

        x_str = entry_x.get().strip()
        if not x_str:
            messagebox.showwarning("Увага", "Введіть значення x!")
            return

        try:

```

```

        x = float(x_str)
    except ValueError:
        messagebox.showwarning("Увага", "Некоректне числове значення x!")
        return

    if abs(x - key_value[0]) < 1e-12:
        messagebox.showinfo(
            "Завершення",
            f"Введено ключове значення x = {x}.\nПрограма завершує роботу.\n"
            f"Всього обчислень: {len(history)}",
        )
        root.destroy()
        return

    is_degrees = var_degrees.get()
    if is_degrees:
        x_calc = deg_to_rad(x)
        input_info = f"x = {x}° ({x_calc:.6f} рад)"
    else:
        x_calc = x
        input_info = f"x = {x}"

    try:
        y, subfunc = compute_y(x_calc)
        result = f"{input_info} → {subfunc} → y = {y:.10f}"
    except ValueError as e:
        result = f"{input_info} → Помилка: {e}"

    history.append(result)
    text_history.config(state="normal")
    text_history.insert(tk.END, f"{len(history)}. {result}\n")
    text_history.see(tk.END)
    text_history.config(state="disabled")
    entry_x.delete(0, tk.END)

root = tk.Tk()
root.title("Завдання 2 – Варіант 2 (цикл з ключовим значенням)")
root.geometry("700x450")
root.resizable(False, False)

frame = tk.Frame(root, padx=15, pady=10)
frame.pack(fill=tk.BOTH, expand=True)

tk.Label(frame, text="Задайте ключове значення x:").grid(
    row=0, column=0, sticky="w"
)
entry_key = tk.Entry(frame, width=15)
entry_key.grid(row=0, column=1, padx=5)
btn_set_key = tk.Button(frame, text="Встановити", command=set_key)
btn_set_key.grid(row=0, column=2, padx=5)
lbl_key_status = tk.Label(frame, text="", fg="green")
lbl_key_status.grid(row=0, column=3, sticky="w")

tk.Label(frame, text="Введіть значення x:").grid(row=1, column=0, sticky="w", pady=(10, 0))
entry_x = tk.Entry(frame, width=15, state="disabled")
entry_x.grid(row=1, column=1, padx=5, pady=(10, 0))

```

```

btn_calc = tk.Button(frame, text="Обчислити", command=calculate, state="disabled")
btn_calc.grid(row=1, column=2, padx=5, pady=(10, 0))

var_degrees = tk.BooleanVar(value=False)
tk.Checkbutton(
    frame,
    text="Аргумент у градусах",
    variable=var_degrees,
).grid(row=2, column=0, columnspan=2, sticky="w", pady=5)

tk.Label(frame, text="Історія обчислень:").grid(
    row=3, column=0, columnspan=4, sticky="w"
)
text_history = tk.Text(frame, height=15, width=85, state="disabled", font=("Consolas", 9))
text_history.grid(row=4, column=0, columnspan=4, pady=5)

scrollbar = tk.Scrollbar(frame, command=text_history.yview)
scrollbar.grid(row=4, column=4, sticky="ns")
text_history.config(yscrollcommand=scrollbar.set)

root.mainloop()

if __name__ == "__main__":
    run_gui()

```

4.3 Результати тестування

Сценарій тестування:

1. Запуск програми → з'являється вікно з полем для ключового значення.
2. Вводимо ключове значення: 999 → поле блокується, активується поле x.
3. Вводимо $x = 0.5$ → результат: $y = -0.5369857229$
4. Вводимо $x = 4.0$ → результат: $y = 0.4918043530$
5. Вводимо $x = 2.0$ → повідомлення про помилку ($x = 2$ не визначена)
6. Вводимо $x = 999$ → повідомлення про завершення, вікно закривається.

Програма коректно реалізує циклічну обробку з ключовим значенням. Історія всіх обчислень відображається у текстовому полі з прокруткою.

5. Завдання 3: Обчислення суми ряду

5.1 Постановка задачі

Обчислити суму n елементів ряду (Варіант 2):

$$S = \cos(x-n-1)/\sin(x) - \sin(x-n-2)/\cos(x^3) + \cos(x-n-3)/\sin(x^5) - \dots$$

Загальний член ряду ($k = 1, 2, \dots, n$):

- Знак: $(-1)^{(k+1)}$
- Чисельник: $\cos(x-n-k)$ якщо k непарне, $\sin(x-n-k)$ якщо k парне
- Знаменник: $\sin(x^{(2k-1)})$ якщо k непарне, $\cos(x^{(2k-1)})$ якщо k парне

Програма містить 3 функції: for, while, рекурсія.

Числа $n \in \mathbb{N}$ та $x \in \mathbb{R}$ вводяться з клавіатури.

5.2 Код програми (task3.py)

```
"""
```

Лабораторна робота №1. Завдання 3. Варіант 2.

Обчислення суми n елементів ряду:

$$S = \cos(x-n-1)/\sin(x) - \sin(x-n-2)/\cos(x^3) + \cos(x-n-3)/\sin(x^5) - \dots$$

Три функції: for, while, рекурсія.

Кожна повертає (success: bool, result: float).

$n \in \mathbb{N}$, $x \in \mathbb{R}$.

```
"""
```

```
import math
```

```
def term(k, x, n):
```

```
    """
```

Обчислює k -й член ряду (k від 1).

Знак: $(-1)^{(k+1)}$

Чисельник: $\cos(x-n-k)$ якщо k непарне, $\sin(x-n-k)$ якщо k парне

Знаменник: $\sin(x^{(2k-1)})$ якщо k непарне, $\cos(x^{(2k-1)})$ якщо k парне

```
    """
```

```
    sign = (-1) ** (k + 1)
```

```
    arg_num = x - n - k
```

```
    if k % 2 == 1:
```

```
        numerator = math.cos(arg_num)
```

```
    else:
```

```
        numerator = math.sin(arg_num)
```

```
    power = 2 * k - 1
```

```
    x_pow = x ** power
```

```
    if k % 2 == 1:
```

```

        denominator = math.sin(x_pow)
    else:
        denominator = math.cos(x_pow)

    if abs(denominator) < 1e-15:
        raise ValueError(
            f"Знаменник дорівнює нулю у члені k={k}: "
            f"'{'sin' if k % 2 == 1 else 'cos'}(x^{power}) = 0"
        )

    return sign * numerator / denominator


def sum_for(n, x):
    """Обчислення суми за допомогою циклу for."""
    try:
        s = 0.0
        for k in range(1, n + 1):
            s += term(k, x, n)
        return True, s
    except (ValueError, OverflowError, ZeroDivisionError) as e:
        return False, str(e)


def sum_while(n, x):
    """Обчислення суми за допомогою циклу while."""
    try:
        s = 0.0
        k = 1
        while k <= n:
            s += term(k, x, n)
            k += 1
        return True, s
    except (ValueError, OverflowError, ZeroDivisionError) as e:
        return False, str(e)


def sum_recursive(n, x, k=1, acc=0.0):
    """Обчислення суми за допомогою рекурсії."""
    try:
        if k > n:
            return True, acc
        acc += term(k, x, n)
        return sum_recursive(n, x, k + 1, acc)
    except (ValueError, OverflowError, ZeroDivisionError) as e:
        return False, str(e)


def main():
    print("=" * 60)
    print("Завдання 3 – Варіант 2")
    print("Обчислення суми ряду:")
    print("S = cos(x-n-1)/sin(x) - sin(x-n-2)/cos(x³)")
    print("      + cos(x-n-3)/sin(x⁵) - ...")
    print("=" * 60)

```

```

while True:
    try:
        n = int(input("\nВведіть n (натуральне число): "))
        if n <= 0:
            print("n повинно бути натуральним числом (> 0)!")
            continue
        break
    except ValueError:
        print("Некоректне значення! Введіть ціле число.")

while True:
    try:
        x = float(input("Введіть x (дійсне число): "))
        break
    except ValueError:
        print("Некоректне значення! Введіть число.")

print(f"\nВхідні дані: n = {n}, x = {x}")
print("-" * 60)

success_for, result_for = sum_for(n, x)
if success_for:
    print(f"Цикл FOR: S = {result_for:.10f}")
else:
    print(f"Цикл FOR: Помилка – {result_for}")

success_while, result_while = sum_while(n, x)
if success_while:
    print(f"Цикл WHILE: S = {result_while:.10f}")
else:
    print(f"Цикл WHILE: Помилка – {result_while}")

success_rec, result_rec = sum_recursive(n, x)
if success_rec:
    print(f"Рекурсія: S = {result_rec:.10f}")
else:
    print(f"Рекурсія: Помилка – {result_rec}")

if success_for and success_while and success_rec:
    print("-" * 60)
    print("Усі три методи дали однаковий результат:", end=" ")
    if abs(result_for - result_while) < 1e-12 and abs(result_for - result_rec) < 1e-12:
        print("ТАК")
    else:
        print("НІ (можливі похибки округлення)")

print()

if __name__ == "__main__":
    main()

```

5.3 Результати тестування

n	x	FOR	WHILE	Рекурсія
5	2.0	4.8350074528	4.8350074528	4.8350074528
3	1.5	-0.6607358355	-0.6607358355	-0.6607358355
10	0.7	281.1704618708	281.1704618708	281.1704618708
4	3.0	-3.7632434335	-3.7632434335	-3.7632434335

Усі три методи (for, while, рекурсія) дають ідентичні результати, що підтверджує коректність реалізації. Функції повертають кортеж (success: bool, result), де success = True при успішному обчисленні і False при виникненні помилки (ділення на нуль, переповнення тощо).

6. Висновки

У ході виконання лабораторної роботи було:

1. Розроблено програму обчислення складної функції (Варіант 2) з повною перевіркою області визначення, графічним інтерфейсом tkinter та підтримкою введення аргументів у градусах.
2. Модифіковано програму для циклічної роботи з "ключовим" значенням x , яке задається користувачем. Програма веде історію обчислень та завершується при введенні ключового значення.
3. Реалізовано обчислення суми ряду трьома способами: циклом for, циклом while та рекурсією. Всі три функції повертають логічне значення успішності та результат. Тестування підтвердило ідентичність результатів усіх трьох методів.

Усі програми написані мовою Python з використанням модулів math та tkinter. Реалізовано обробку всіх можливих помилок (ділення на нуль, вихід за область визначення функцій тощо).